

FACULTAD DE INGENIERÍA
UNIVERSIDAD NACIONAL DE ENTRE RÍOS
BIOINGENIERÍA



Trabajo Práctico N° 1 - Aplicaciones de TADs

INTEGRANTE/S:

Muller González Brian Ezequiel,
Velazco Francisco,
Yang Luis Fernando

FECHA DE ENTREGA: 04 de Mayo del 2026

LINK DEL REPOSITORIO:

<https://github.com/Brianmuller26/AyED2026c1-Muller-Velazco-Yang.git>

INTRODUCCIÓN

El siguiente trabajo práctico tiene como objetivo implementar Tipos Abstractos de Datos (TADs) y relacionar conceptos necesarios para la implementación de algoritmos de ordenamiento.

También se analiza la complejidad (notación O-grande) de los algoritmos empleados, así como el manejo de excepciones, implementación de pruebas unitarias (tests) para corroborar el correcto funcionamiento del código realizado, y el uso de precondiciones y postcondiciones.

Además, se analizaron las operaciones críticas involucradas en cada problema, considerando eficiencia temporal y comportamiento en distintos casos de ejecución.

El trabajo también permitió afianzar conceptos relacionados con manejo de excepciones, organización de proyectos en módulos y utilización de herramientas colaborativas como Git y GitHub para el desarrollo en equipo.

DESARROLLO DE LOS PROBLEMAS

Problema 1

Análisis y planteo

El problema consiste en implementar una Lista Doblemente Enlazada capaz de almacenar elementos dinámicamente permitiendo inserciones, eliminaciones y recorridos en ambos sentidos.

Las operaciones críticas del problema son la inserción y eliminación de nodos, especialmente en posiciones extremas y posiciones arbitrarias de la estructura. Debido a esto, una lista doblemente enlazada resulta adecuada ya que cada nodo mantiene referencias tanto al nodo siguiente como al anterior, permitiendo recorridos bidireccionales y facilitando determinadas operaciones respecto a una lista simplemente enlazada.

La elección de esta estructura es razonable para conjuntos de datos dinámicos donde las modificaciones son frecuentes y el tamaño de la estructura puede variar durante la ejecución.

Implementación / Resolución

La solución se desarrolló utilizando una clase Nodo y una clase ListaDobleEnlazada. Cada nodo almacena:

- El dato contenido.
- Una referencia al nodo siguiente.
- Una referencia al nodo anterior.

La lista mantiene referencias a:

- Cabeza.
- Cola.
- Tamaño de la estructura.

Se implementaron métodos para:

- Agregar elementos.
- Eliminar nodos.
- Recorrer la lista.
- Consultar elementos.
- Verificar si la lista está vacía.

Resultados y pruebas

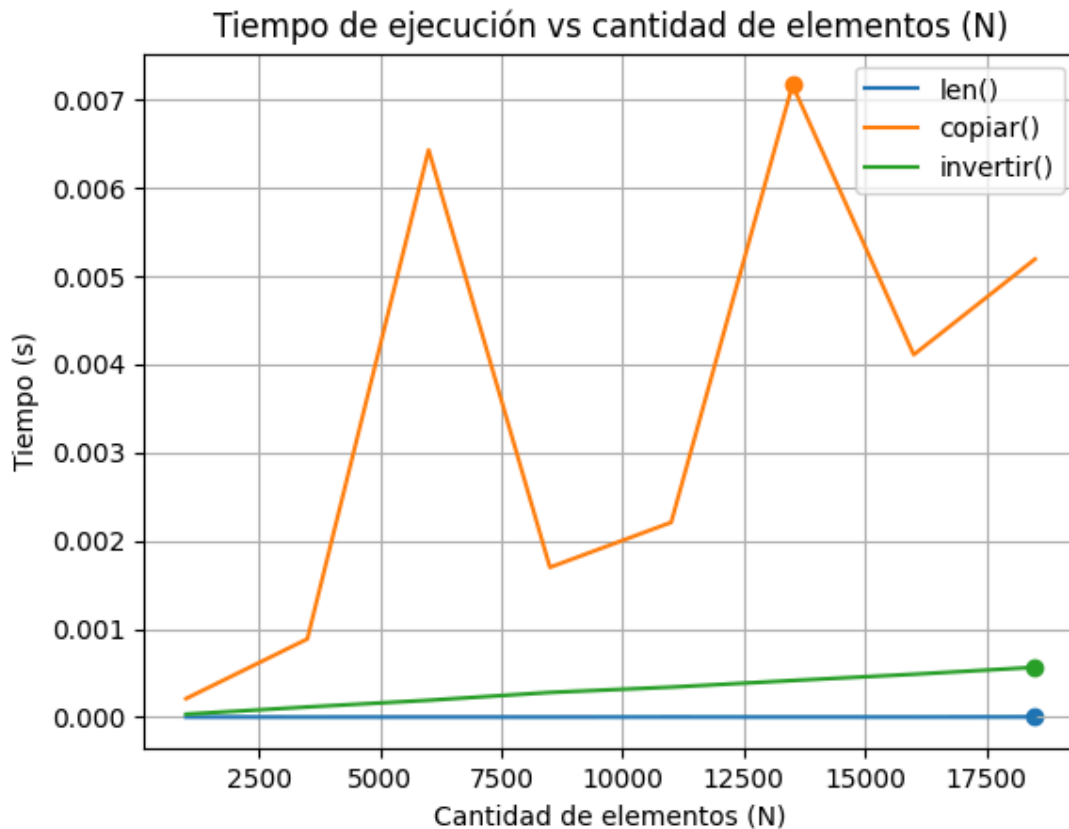
Para validar el funcionamiento de la estructura se realizaron pruebas unitarias utilizando distintos casos de prueba.

Se verificó:

- Inserción correcta de elementos.
- Eliminación en cabeza y cola.
- Funcionamiento con lista vacía.
- Actualización correcta de referencias anteriores y siguientes.
- Correcto mantenimiento del tamaño de la lista.

Los resultados obtenidos fueron satisfactorios y permitieron comprobar que la estructura funciona de acuerdo con las especificaciones planteadas.

En la siguiente gráfica se puede observar los resultados de ejecución del tiempo para los métodos:



A partir de la gráfica, podemos deducir los siguientes órdenes de complejidad:

- `len()` — Complejidad $O(1)$: el tiempo de ejecución no aumenta sin importar la cantidad de elementos. Esto sucede porque el método no recorre la lista para contar, sino que accede a un atributo interno (`self.__tamano`) previamente actualizado en cada inserción o eliminación.
- `invertir()` — Complejidad $O(n)$: el tiempo crece de manera directamente proporcional a n . El algoritmo debe chequear cada nodo exactamente una vez para intercambiar sus punteros siguiente y anterior, por lo tanto se ve un comportamiento lineal.
- `copiar()` — Complejidad $O(n)$: al igual que la inversión, copiar requiere iterar sobre los n elementos para instanciar nuevos nodos. La implementación es eficiente porque se añaden elementos al final de la nueva lista en tiempo constante, evitando el costo de insertar por posición.

Los picos que se ven en la función de `copiar()` es porque en la memoria se almacena basura y no logra eliminarlo (podemos decir que es ruido digital).

Problema 2

Análisis y planteo

El problema consiste en modelar el comportamiento de un juego de cartas entre dos jugadores, donde cada uno dispone de un mazo propio y las operaciones principales implican extraer cartas del inicio y agregarlas al final. Podemos interpretar este problema como la manipulación de dos secuencias dinámicas de elementos (cartas), donde se realizan operaciones de tipo cola/doble extremo (deque).

Las operaciones críticas podemos identificarlas al leer el problema y entender cuáles son las acciones que pueden hacer los jugadores en cada turno. Teniendo esto en cuenta, entonces podemos decir que tenemos:

- Extracción del primer elemento (sacar la carta superior del mazo).
- Inserción al final (agregar cartas ganadas al final del mazo).
- Inserción al inicio (utilizada al repartir cartas).
- Consulta del tamaño (para determinar si un jugador se queda sin cartas).

También debemos manejar la excepción `DequeEmptyError` en caso de intentar extraer una carta de un mazo vacío.

Dado este conjunto de operaciones y habiendo resuelto el problema 1, queda claro que la estructura de datos más adecuada es una lista doblemente enlazada, ya que podremos usar sus métodos de agregar al inicio y al final (insertar y extraer en los extremos) con una complejidad $O(1)$.

A partir de esto, definimos una clase `Mazo` que utiliza nuestra LDE del problema 1.

Implementación / Resolución

La solución que planteamos es la implementación de la clase `Mazo`, la cual internamente contiene una instancia de la `ListaDobleEnlazada` del problema 1, y un atributo `__tamano` para controlar el número de cartas en el mazo.

Luego, respecto a las funciones que debe realizar esta clase tenemos las inserciones en `poner_carta_arriba` y `poner_carta_abajo`, y por último la extracción en `sacar_carta_arriba`, en la cual también debemos hacer uso de la excepción `DequeEmptyError` para indicar que no es posible extraer cartas de un mazo vacío. Esta excepción la definimos en una clase aparte para luego utilizarla.

Resultados y pruebas

Para verificar que la solución implementada funciona utilizamos los tests provistos por la cátedra. El objetivo de estos tests es, por un lado, probar que nuestra clase `Mazo` funcione, y por otro que también sea adecuada para que de manera global el juego de guerra funcione.

Para las pruebas sobre nuestra clase `Mazo`, se verificó:

- Inserción y extracción al inicio: se agregan cartas al inicio del mazo y luego que la extracción respete el orden LIFO.
- Inserción al final del mazo: se agregan cartas al final del mazo y luego al extraer desde el inicio se respete el orden FIFO.

En cuanto al juego completo, utilizamos el test juego de guerra el cual verifica:

- La cantidad de turnos jugados.
- El jugador ganador.
- Empate si es que ocurre.

Que este último test funcione nos indica que el juego se desarrolla de manera correcta y que las operaciones que nosotros planteamos sobre los mazos son consistentes a lo largo de todos los turnos de la partida.

Finalmente y como ya mencionamos, hay un caso límite que ocurre si intentamos extraer una carta de un mazo vacío; en este contexto, se produce la excepción `DequeEmptyError`.

Problema 3

Análisis y planteo

El problema consiste en implementar y comparar empíricamente tres algoritmos de ordenamiento (Burbuja, Quicksort y Radix Sort) para evaluar su eficiencia en términos de tiempo de ejecución y complejidad algorítmica.

Las operaciones críticas del problema son la comparación de elementos, el intercambio de posiciones en la lista, el particionamiento de subconjuntos de datos y la distribución de elementos basada en sus dígitos. Debido a esto, se utilizan diferentes estrategias: un enfoque iterativo simple para Burbuja, un enfoque de "divide y vencerás" para Quicksort y un enfoque de ordenamiento no comparativo para Radix Sort.

Implementación / Resolución

Cada algoritmo se implementó siguiendo su lógica fundamental:

- Ordenamiento Burbuja: utilizamos dos bucles anidados para comparar elementos adyacentes e intercambiarlos si están fuera de orden.
- Quicksort: implementamos una función recursiva que selecciona un pivote y organiza los elementos en sublistas de menores y mayores.
- Radix Sort: procesamos los números de cinco dígitos en 5 pasadas, distribuyéndolos en "baldes" según el valor de cada posición decimal.

Para la medición y comparación, se implementaron procesos para:

- Generar listas de números aleatorios de cinco dígitos (entre 10,000 y 99,999).
- Cronometrar el tiempo de ejecución mediante la librería `time`.
- Graficar los resultados utilizando `matplotlib` para visualizar las curvas de rendimiento.

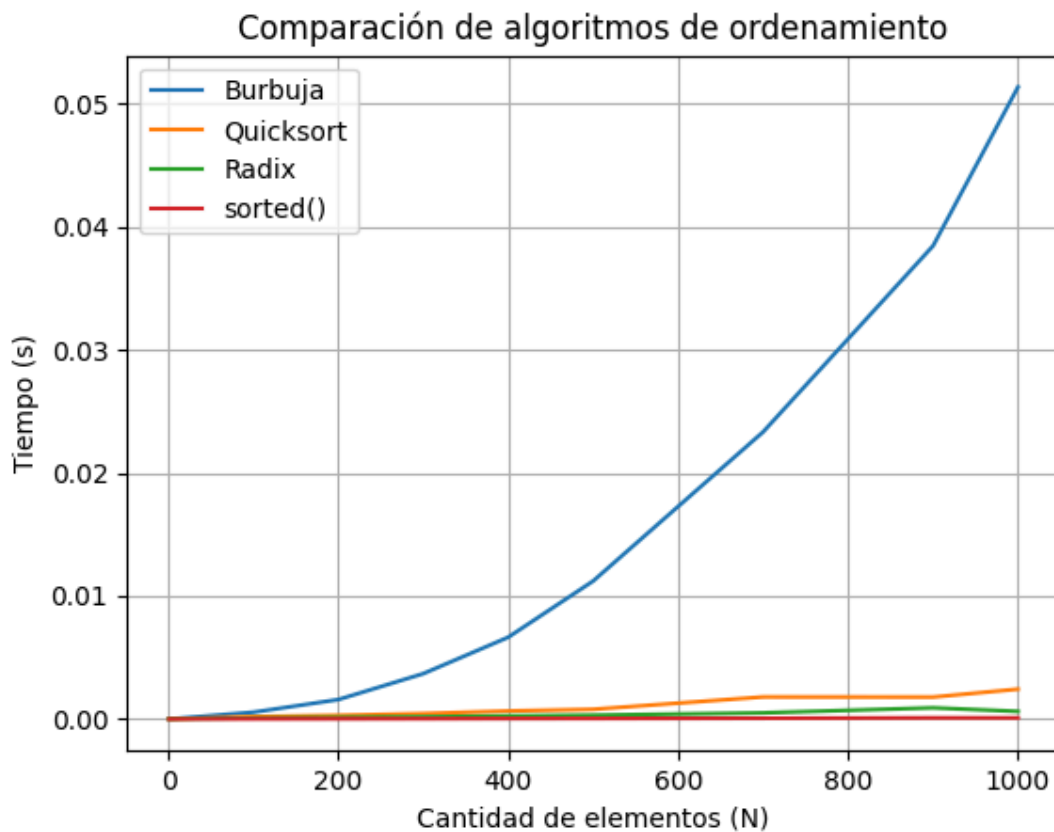
Respondiendo a la pregunta del algoritmo Sorted, este utiliza un algoritmo llamado Timsort. Es un híbrido derivado del Merge Sort y el Insertion Sort. Está diseñado para aprovechar secuencias ya ordenadas en datos reales, lo que le otorga una complejidad de $O(n \log n)$ en el peor caso, pero un desempeño cercano a $O(n)$ en datos parcialmente ordenados.

Resultados y pruebas

Pudimos corroborar que los tres algoritmos aplicados a listas de 500 números produjeron resultados idénticos a la función Sorted.

Se justificó la complejidad cuadrática $O(n^2)$ de Burbuja por sus bucles anidados, la complejidad promedio $O(n \log n)$ de Quicksort por su división logarítmica, y la linealidad $O(n)$ de Radix Sort para un número fijo de dígitos.

En el gráfico se puede ver que las curvas obtenidas mostraron que Burbuja crece de forma parabólica, mientras que Quicksort y Radix mantienen una pendiente mucho más baja, siendo sorted el más eficiente debido a su implementación optimizada.



ANÁLISIS DE RESULTADOS

Durante el desarrollo del trabajo se observó que las estructuras enlazadas permiten una mayor flexibilidad para inserciones y eliminaciones respecto a estructuras contiguas como las listas tradicionales de Python. Sin embargo, también presentan un mayor consumo de memoria debido al almacenamiento de referencias adicionales en cada nodo.

En relación con los algoritmos de ordenamiento implementados, se verificó que algoritmos simples como el burbuja poseen una complejidad temporal elevada en listas grandes, lo que afecta significativamente el rendimiento en comparación con métodos más eficientes.

USO ÉTICO DE INTELIGENCIA ARTIFICIAL

Herramientas utilizadas:

- Chat GPT

Descripción del uso:

- Asistencia para redactar explicaciones técnicas y mejorar la claridad del texto.
- Consulta teórica sobre complejidad computacional y algoritmos de ordenamiento.
- Ayuda en depuración de errores sintácticos y validación lógica de algunas funciones.
- Ayuda para la implementación del ploteo de gráficas mediante la librería matplotlib.

Declaración de responsabilidad:

Los integrantes del grupo declaramos que hemos revisado y validado personalmente toda la información y código sugerido por las herramientas de IA. Entendemos los conceptos presentados en este informe y asumimos la responsabilidad académica por la integridad y veracidad del contenido. El trabajo final es producto de nuestro propio análisis y criterio.

TRABAJO EN EQUIPO

El trabajo fue desarrollado de manera colaborativa, distribuyendo las distintas tareas entre según las necesidades de cada etapa del proyecto, pero siempre teniendo en cuenta de estar todos juntos a la hora de plantear las soluciones para que entendamos lo que hacemos como grupo. Nos centramos en diseñar e implementar las estructuras de datos solicitadas, asegurando una lógica sólida en sus operaciones y métodos fundamentales.

Para garantizar la calidad y robustez del software, integramos precondiciones y postcondiciones en el código, complementadas con una fase de pruebas y validación que permitió la detección y corrección de errores en tiempo real.

La comunicación y coordinación del trabajo se realizó mediante WhatsApp y GitHub, utilizando el repositorio compartido para integrar los cambios realizados por cada integrante. A medida que avanzaba el desarrollo, se realizaron revisiones grupales del código para comprobar el funcionamiento correcto de las implementaciones y resolver problemas que surgían durante las pruebas.

Durante el desarrollo aparecieron algunas dificultades relacionadas con la integración de cambios en el repositorio y con la implementación de ciertos métodos de las estructuras enlazadas. Estas situaciones fueron resolviéndose mediante pruebas adicionales, revisión conjunta del código y ajustes progresivos hasta alcanzar una versión funcional del trabajo práctico.

CONCLUSIONES

Los objetivos planteados fueron cumplidos satisfactoriamente, logrando implementar distintas estructuras y algoritmos aplicando conceptos fundamentales de programación y análisis computacional.

Entre las principales dificultades encontradas se destacaron la correcta manipulación de referencias entre nodos y el manejo de casos límite en las estructuras enlazadas. Estas dificultades pudieron resolverse mediante pruebas unitarias y depuración progresiva del código.

El trabajo permitió comprender de manera práctica la importancia de elegir estructuras de datos adecuadas según las operaciones requeridas y reforzó el uso de herramientas colaborativas para el desarrollo en equipo.

BIBLIOGRAFÍA

Laboratorio de Informática y Computación Aplicada (LICA). (2026). *Precondiciones, Postcondiciones e Invariantes: de los límites de la computación a la especificación de algoritmos*. Facultad de Ingeniería, Universidad Nacional de Entre Ríos (UNER).
<https://docs.google.com/document/d/1pO310W2b2Qj0Kq8uzstduWqkdRepy2GH4QSO8i1aXps/edit?tab=t.0>

Miller, B., & Ranum, D. (s.f.). *Solución de problemas con algoritmos y estructuras de datos usando Python*. Luther College.
<https://runestone.academy/ns/books/published/pythoned/index.html>